CE 502 · ADVANCED TOPICS IN DATABASE SYSTEMS

Week 05

Horizontal Fragmentation

How to cut a table into groups of rows, so that each piece is stored where it is used and still adds up to the whole table.

Dr. Afan HASAN · [Semester]

THIS WEEK

- **5-1** Distributed Design and Correctness — 45 minutes
- **5-2** Primary Horizontal Fragmentation — 45 minutes
- **5-3** Derived Horizontal Fragmentation — 45 minutes
- **P5** Practice: fragments as partitions — 60 minutes · PostgreSQL lab

Recap: Distributed Databases and Their Algebra

- A **distributed database** is one logical database stored at several sites and managed so that users see one database.
- **Fragmentation transparency:** queries name the global relation; the DBMS maps them to fragments.
- **σ** picks rows, **π** picks columns, **∪** and **⋈** combine; **R ⋉ S** keeps the rows of R that match S.

THIS WEEK THE OPERATORS DEFINE FRAGMENTS

- **σ** **Selection** defines a horizontal fragment.
- **∪** **Union** rebuilds the relation.
- **⋉** **Semi-join** derives a related relation's fragments.

Key point: Week 4 assumed the data was already distributed. This week you decide how to distribute it.

CE 502 · WEEK 5 · LESSON

5-1

Distributed Design and Correctness

Lesson 1 of 3

In this lesson

01 Where fragmentation fits in distribution design

02 Why a whole table is the wrong unit of distribution

03 Horizontal, vertical and hybrid fragments

04 Three rules every fragmentation must pass

45 minutes

Builds on Week 4: σ, π, ∪, ⋈

Objectives

By the end of this lesson, you will be able to:

01 **Describe** the top-down design process and where fragmentation happens

Checked in: Quiz 5 · question 1

02 **Explain** why relations are fragmented and what fragmentation costs

Checked in: Quiz 5 · question 2

03 **Distinguish** horizontal, vertical and hybrid fragmentation and how each is rebuilt

Checked in: Quiz 5 · questions 3–4

04 **Apply** the completeness, reconstruction and disjointness rules to a design

Checked in: Quiz 5 · questions 5–6 and the scenario

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 01

Two Design Questions for a Distributed Database

- In Week 4 the data was already distributed. Now **you** decide how to distribute it.
- **Fragmentation** decides how each global relation is cut into pieces, called fragments.
- **Allocation** decides at which site each fragment is stored, and whether it is copied.

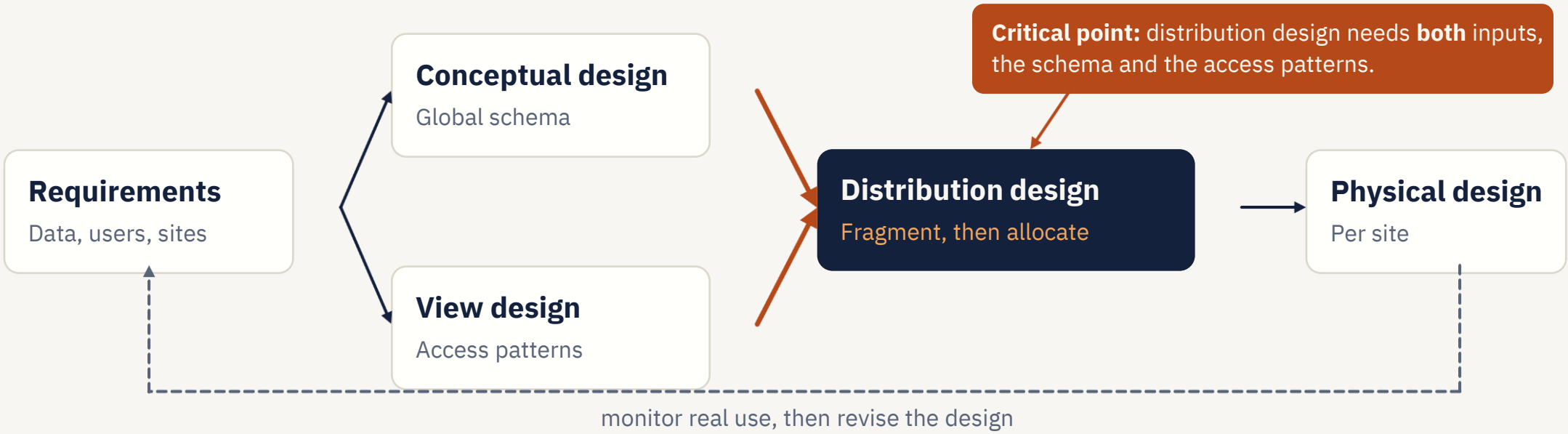
This week and next

- Week 5: cut by **rows**
- Week 6: cut by **columns**
- Then: place the fragments

Key point: fragmentation and allocation are separate decisions. First decide **what** the pieces are, then decide **where** they go.

Objective 01

Top-Down Design: Where Fragmentation Happens



- Conceptual design and view design run **in parallel** after requirements analysis.
- Distribution design maps the global schema to **local schemas** at each site.

Objective 01

Top-Down and Bottom-Up Design

Top-down

When: a new system is designed from scratch.

Start: one global schema, then fragments and sites.

Typical: a company-wide system for several offices.

This week: the approach we use.

Bottom-up

When: several databases already exist.

Start: local schemas, then one global schema on top.

Typical: merging the databases of acquired companies.

Also called: database integration, multidatabase systems.

⚠ **Common mistake:** thinking bottom-up design involves fragmentation. In bottom-up design the data is already placed; the problem is to **integrate** schemas, not to cut relations.

Objective 02

Why Not Distribute Whole Relations?

- Applications rarely use a whole relation: each office reads **its own** projects.
- Store the whole relation at one site, and every other office works **remotely**.
- Copy it to every site, and every update must reach **every** copy.
- A **fragment** matches what one application actually uses.

What fragments give you

- Locality: data near its users
- Parallelism across sites
- Smaller scans per query
- Sensitive data kept apart

Key point: the right unit of distribution is a **subset** of a relation that one group of applications uses together.

Objective 02

The Price of Fragmentation

Situation	What happens	Cost
A query needs rows from two fragments	Results are combined with a union across sites	Network transfer
A query joins attributes in two vertical fragments	Fragments are joined across sites	Remote join
A rule spans fragments, e.g. a foreign key	The check must ask another site	Slower updates
Access patterns change	Fragments no longer match the queries	Redesign

In practice

Systems such as Citus and CockroachDB report the share of **single-site** transactions. A good design keeps that share high.

← **Read this row first.** A bad cut turns every frequent query into a cross-site union.

⚠ **Common mistake:** assuming more fragments are always better. Each extra fragment is one more piece that some query may have to fetch and combine.

Objective 03

Two Ways to Cut a Relation

PROJ(**pno**, pname, budget, loc), the projects of the engineering company.

Horizontal: by rows

$PROJ_1 = \sigma$ budget ≤ 200,000 (PROJ)

$PROJ_2 = \sigma$ budget > 200,000 (PROJ)

Same columns, different rows.

Rebuild: $PROJ = PROJ_1 \cup PROJ_2$

Vertical: by columns

$PROJ_1 = \pi$ pno, budget (PROJ)

$PROJ_2 = \pi$ pno, pname, loc (PROJ)

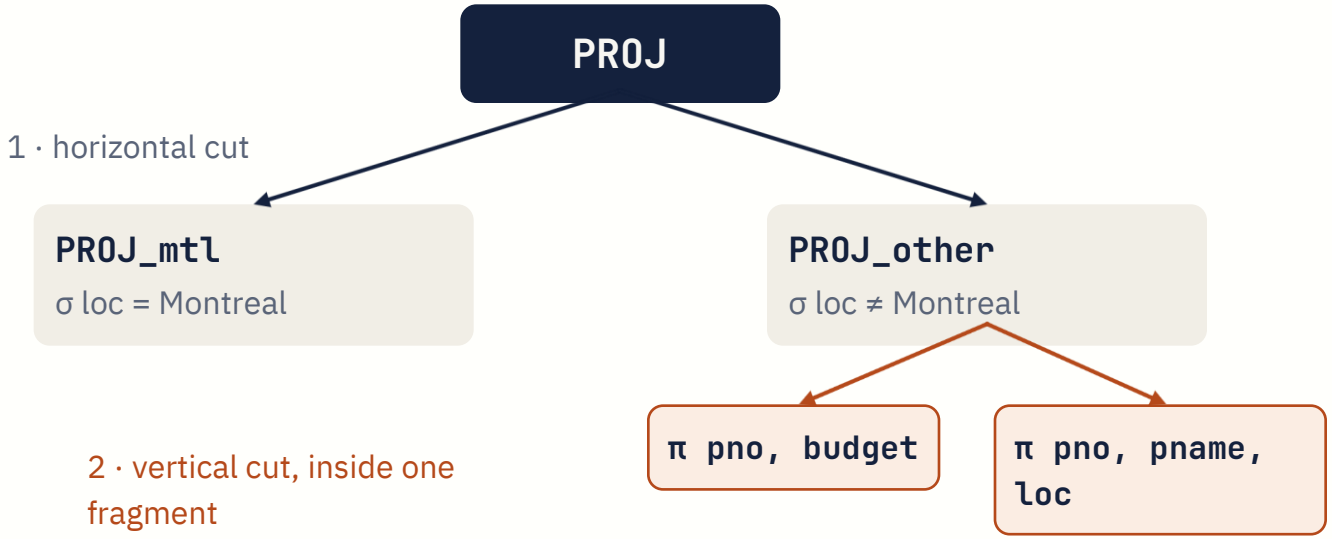
Every fragment keeps the key **pno**.

Rebuild: $PROJ = PROJ_1 \bowtie PROJ_2$ on pno

→ **Why keep pno twice?** Without the key in both fragments, the join cannot match the pieces again.

Objective 03

Hybrid Fragmentation: Cuts Inside Cuts



- A **hybrid** (nested) fragmentation applies one kind of cut inside a fragment of the other kind.
- Rebuild from the leaves up: **join** the vertical pieces, then **union** the horizontal ones.

→ **Note:** each step must pass the correctness rules on its own.

Objective 04

Three Rules Every Fragmentation Must Pass

Whatever kind of cut you choose, these three tests decide whether the design is correct.

Rule	Meaning	Horizontal	Vertical
Completeness	Every item of R, now and in the future, is in some fragment	Every row satisfies some predicate	Every column is in some fragment
Reconstruction	An operator rebuilds R from its fragments	Union ∪	Join ⋈ on the key
Disjointness	No item is in two fragments	No row in two fragments	No non-key column twice

Read this row first.
Completeness must also hold for rows that do not exist yet.

In practice

PostgreSQL refuses an insert that fits no partition unless a **DEFAULT** partition exists. You will see this in Practice 5.

Objective 04

Lose nothing, rebuild everything, store nothing twice.

Completeness, reconstruction and disjointness are the three tests of a correct fragmentation. Replicas are copies made later, during allocation, and do not break disjointness.

Rule of thumb: test every new fragmentation with one row that does not exist yet.

Objective 04

Scenario: Three Proposals for PROJ

THE SITUATION

The engineering company asks three junior designers to fragment PROJ(pno, pname, budget, loc). Their proposals:

A · σ budget < 100,000 and σ budget > 200,000

B · σ budget ≤ 200,000 and σ budget ≥ 150,000

C · π pno, budget and π pname, loc

"All three look fine to me. Which one do we use?"

YOUR TASK · PAIRS · 8 MIN

1. Which correctness rule does each proposal break?
2. Give one project row that shows the problem for A and for B.
3. Repair each proposal with the smallest change.
4. Is any proposal correct as it stands?

Check Your Understanding

EMP(eno, ename, title, sal) is fragmented vertically into $EMP_1 = \pi$ eno, ename, title and $EMP_2 = \pi$ eno, title, sal. Which statement is true?

A It breaks completeness: sal is missing

B It breaks reconstruction: there is no key

C It breaks disjointness: title is in both

D It is correct: eno is in both fragments

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Fragment
A subset of a relation, defined by selection (rows) or projection (columns).

Allocation
The decision of which site stores each fragment, and whether it is copied.

Top-down design
Design from requirements to a global schema, then fragments and sites.

Bottom-up design
Integration of existing databases under one global schema.

Horizontal fragmentation
Cutting a relation by rows; rebuilt with union.

Vertical fragmentation
Cutting a relation by columns, keeping the key; rebuilt with join.

Completeness
Every row or column of the relation is in some fragment.

Disjointness
No row, or no non-key column, is in two fragments.

Summary

In this lesson, you learned how to:

- **01** Describe top-down design and where fragmentation happens
- **02** Explain why relations are fragmented and what it costs
- **03** Distinguish horizontal, vertical and hybrid fragmentation
- **04** Apply completeness, reconstruction and disjointness

Key takeaway: fragment a relation so that each piece matches what one group of applications uses, and check every design against the three rules.

Next lesson

5-2 Primary Horizontal Fragmentation

CE 502 · WEEK 5 · LESSON

5-2

Primary Horizontal Fragmentation

Lesson 2 of 3

In this lesson

01 The goal: rows in a fragment are used alike

02 Simple predicates, minterms and implications

03 Complete and minimal predicate sets

04 The five-step algorithm, worked on PROJ

45 minutes

Builds on 5-1: the three rules

Objectives

By the end of this lesson, you will be able to:

01 **Explain** the goal of horizontal fragmentation in terms of access patterns

Checked in: Quiz 5 · question 7

02 **Identify** simple predicates and test a set for completeness and minimality

Checked in: Quiz 5 · questions 8–9

03 **Build** minterms and remove contradictory ones using implications

Checked in: Quiz 5 · questions 10–11

04 **Apply** the five-step algorithm and check the resulting fragments

Checked in: Quiz 5 · question 12 and Practice 5

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 01

The Goal: Rows in a Fragment Are Used Alike

- Inside one fragment, every row should be **equally likely** to be read by each application.
- Rows that applications treat **differently** belong in different fragments.
- One row per fragment meets the goal, but every query becomes a huge union: **correct but useless**.

Test for two candidate fragments

$acc(m_i) / card(f_i)$

accesses per row in fragment i

$acc(m_j) / card(f_j)$

accesses per row in fragment j

Clearly different: keep them apart. Nearly equal: merge them.

Key point: a fragment is right when no application could tell its rows apart by how often it uses them.

Objective 02

What You Need to Know Before Fragmenting

Database information

The global schema: relations and attributes.

The **links** between relations: owner (1 side) and member (N side).

The size of each relation: card(R).

Value domains: which values an attribute can take.

Application information

Simple predicates used in the applications' queries.

Selectivity: how many rows each minterm selects.

Access frequency of each application and minterm.

Where each application runs.

In practice

A common rule of thumb: about 20% of the applications cause about 80% of the accesses. Analyse those first, from the query log or the workload statistics.

Objective 02

Simple Predicates and Minterms

Simple predicate

attribute θ value, with θ one of =, <, ≤, >, ≥, ≠.

Taken from the queries of real applications.

p_1: loc = 'Paris'

p_2: budget ≤ 200,000

Minterm

A conjunction of **every** simple predicate, each **as-is or negated**.

n predicates give $\mathbf{2^n}$ minterms.

$p_1 \wedge p_2 \cdot p_1 \wedge \neg p_2 \cdot \neg p_1 \wedge p_2 \cdot \neg p_1 \wedge \neg p_2$

Each minterm can define one fragment.

Note: a minterm never leaves a predicate out. That is why minterm fragments can never overlap.

Objective 02

Complete and Minimal Predicate Sets

Complete

Inside every fragment, each row has the **same probability** of being accessed by every application.

Missing predicate: one fragment still mixes rows used often and rows used rarely.

Fix: add the predicate that separates them.

Minimal

Every predicate is **relevant**: it splits some fragment into parts that are accessed **differently**.

Extra predicate: two fragments with the same access rate per row.

Fix: drop the predicate and merge them.

⚠ **Common mistake:** confusing a complete **predicate set** with the **completeness rule**. The rule says every row has a fragment; a complete set says the fragments are used uniformly.

Objective 03

Implications Remove Impossible Minterms

An implication is a rule between predicates. A minterm that breaks one can never be true, so it is removed.

Implication	Why it holds	Minterms it removes
loc = Montreal ⇒ loc ≠ Paris	An attribute has one value per row	Two or more cities true
¬Montreal ∧ ¬NY ⇒ Paris	The company has only three offices	No city true
budget ≤ 100k ⇒ budget ≤ 200k	Ranges on one attribute are nested	≤ 100k but not ≤ 200k
¬(budget ≤ 200k) ⇔ budget > 200k	Negation of a range	None: it renames the minterm

Domain knowledge: this row is true only because the company has exactly three offices.

In practice

A minterm that is **possible** but has no rows today is kept: future rows will need it.

Objective 04

The Algorithm in Five Steps

Steps 2–5 are mechanical. Step 1 is where the designer's judgement goes.

01

Find predicates from the frequent queries; keep a complete and minimal set

02

Build minterms: all 2^n as-is or negated combinations

03

Write implications between the predicates, from logic and domain knowledge

04

Remove the minterms that the implications make contradictory

05

Define one fragment per remaining minterm, $F_i = \sigma\, m_i\,(R)$, and **check** the three rules

Key point: finding the best fragmentation is NP-hard. This algorithm is a heuristic: a good answer in polynomial time, driven by the designer's choice of predicates.

Worked Example, Step 1: Four Predicates for PROJ

Application 1 · city offices

Each office reads only **its own** projects.

Offices: Montreal, New York, Paris.

The company has no other offices.

Application 2 · head office

Reviews projects with a budget **≤ 200,000** far more often than larger ones.

Runs from Montreal.

SIMPLE PREDICATES

p_1: loc = 'Montreal'

p_2: loc = 'New York'

p_3: loc = 'Paris'

p_4: budget ≤ 200000

→ **Why is p_4 relevant?** Without it, each city fragment mixes rows head office reads often with rows it rarely reads.

Objective 03

Worked Example, Step 2: 16 Minterms, 6 Kept

Steps 2–4: group the 16 minterms by how many location predicates are true.

Pattern of p_1, p_2, p_3	Location patterns	× p_4 or $\neg p_4$	Decision
Two or three cities true	4	8 minterms	Remove: one value per row
No city true	1	2 minterms	Remove: only three offices
Exactly one city true	3	6 minterms	Keep
Total	$8 = 2^3$	$16 = 2^4$	6 fragments

Implications

$p_1 \Rightarrow \neg p_2 \wedge \neg p_3$

$p_2 \Rightarrow \neg p_1 \wedge \neg p_3$

$p_3 \Rightarrow \neg p_1 \wedge \neg p_2$

$\neg p_1 \wedge \neg p_2 \Rightarrow p_3$

Read this row first. Only one city true: these six minterms become the fragments.

Objective 04

Worked Example, Step 3: Six Fragments, Three Sites

Step 5: each remaining minterm becomes a fragment; allocation places it at the office that uses it.

Fragment	Definition	Rows today	Stored at
$PROJ_1$	σ loc = 'Montreal' ∧ budget ≤ 200k (PROJ)	P1	Montreal
$PROJ_2$	σ loc = 'Montreal' ∧ budget > 200k (PROJ)	P6	Montreal
$PROJ_3$	σ loc = 'New York' ∧ budget ≤ 200k (PROJ)	P2	New York
$PROJ_4$	σ loc = 'New York' ∧ budget > 200k (PROJ)	P3	New York
$PROJ_5$	σ loc = 'Paris' ∧ budget ≤ 200k (PROJ)	P5	Paris
$PROJ_6$	σ loc = 'Paris' ∧ budget > 200k (PROJ)	P4	Paris

Try it: a query for small Paris projects reads $PROJ_5$ only. PostgreSQL calls this **partition pruning**.

Complete: every row fits one minterm

Rebuild: union of all six

Disjoint: minterms never overlap

Objective 01

Start from the queries, not from the data.

Every fragment exists because some application uses its rows differently from the rest. A cut that no query uses only adds unions.

Rule of thumb: for every predicate, name the application that needs it. No application, no predicate.

Objective 04

In Practice: Fragments as PostgreSQL Partitions

```
-- level 1: p1–p3 (LIST) · level 2: p4 (RANGE)
1 CREATE TABLE proj_part (LIKE proj)
      PARTITION BY LIST (loc);
2 CREATE TABLE proj_paris PARTITION OF proj_part
      FOR VALUES IN ('Paris') PARTITION BY RANGE (budget);
3 CREATE TABLE proj5 PARTITION OF proj_paris
      FOR VALUES FROM (MINVALUE) TO (200001);
```

1. One partition per location: predicates p_1–p_3.
2. Inside each city, a range split: predicate p_4.
3. $PROJ_5$: Paris and budget ≤ 200,000.

← **Critical:** budget is an integer, so ≤ 200,000 is the range up to **200,001**, which is exclusive.

⚠ **Common mistake:** forgetting a partition for unexpected values. Without a **DEFAULT** partition, an insert for Tokyo fails: the predicate set is not complete for future rows.

Objective 03

Scenario: Fragmenting EMP for Two Offices

THE SITUATION

A software company stores EMP(eno, ename, title, dept). It has only two departments, **Sales** and **R&D**.

Skopje office reads only Sales employees. **Ankara** office reads only R&D employees.

Head office reads **managers** about ten times more often than other employees.

"Give me fragments I can place in Skopje and Ankara."

YOUR TASK · PAIRS · 12 MIN

1. List the simple predicates.
2. How many minterms before elimination?
3. Remove the contradictory minterms. How many remain?
4. Write the fragments with σ.
5. Is your predicate set minimal?

Check Your Understanding

A company has offices only in Montreal and Paris. PROJ is fragmented with p_1: loc = 'Montreal', p_2: loc = 'Paris' and p_3: budget ≤ 200,000. How many fragments remain after contradictory minterms are removed?

A 8

B 6

C 4

D 3

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Primary horizontal fragmentation

Fragmenting a relation with predicates on its own attributes.

Minterm

A conjunction of every simple predicate, each as-is or negated.

Complete predicate set

Rows in each fragment are accessed with the same probability by every application.

Selectivity

The number of rows a minterm selects: the size of its fragment.

Simple predicate

A comparison attribute θ value taken from an application's query.

Implication

A rule between predicates that makes some minterms impossible.

Minimal predicate set

Every predicate is relevant: it splits some fragment into differently accessed parts.

Partition pruning

Reading only the partitions whose bounds match a query's predicates.

Summary

In this lesson, you learned how to:

- **01** Explain the goal: rows in a fragment are used alike
- **02** Identify predicates; test for completeness and minimality
- **03** Build minterms and remove contradictory ones
- **04** Apply the algorithm and check the fragments

Key takeaway: take predicates from the frequent queries, combine them into minterms, remove the impossible ones, and each remaining minterm is a fragment.

Next lesson

5-3 Derived Horizontal Fragmentation

CE 502 · WEEK 5 · LESSON

5-3

Derived Horizontal Fragmentation

Lesson 3 of 3

In this lesson

01 Owners, members and the join graph

02 Deriving fragments with the semi-join

03 Choosing an owner when there are two

04 Correctness of derived fragments

45 minutes

Builds on Week 4: the semi-join ⋉

Objectives

By the end of this lesson, you will be able to:

01 **Describe** owner and member relations and the links of a join graph

Checked in: Quiz 5 · question 13

02 **Apply** the semi-join to derive the fragments of a member relation

Checked in: Quiz 5 · question 14 and Practice 5

03 **Decide** which owner a member with two owners should follow

Checked in: Quiz 5 · question 15 and the design question

04 **Check** derived fragments for completeness, reconstruction and disjointness

Checked in: Quiz 5 · question 16

The chip in the top-right corner of each slide shows which objective that slide serves.

Owners, Members and the Join Graph

Each one-to-many link in the schema connects an owner to a member.



Arrow = link from **owner** (one side) to **member** (many side). Many employees share one pay grade, so PAY owns EMP.

- A relation can be both: EMP is a **member** of PAY and the **owner** of ASG.

Objective 01

How Should EMP Be Fragmented?

- PAY is fragmented by salary: $PAY_1 = \sigma$ sal ≤ 100,000 and $PAY_2 = \sigma$ sal > 100,000.
- Each PAY fragment is stored at a different site.
- The most frequent query on EMP **joins EMP with PAY** on title, for example to print payslips.

EMP has no salary column

A predicate such as sal ≤ 100,000 cannot be written on EMP alone.

Yet the rows of EMP clearly **belong** with one PAY fragment or the other.

Key point: when a relation is mostly read **together with** another relation, cut it by the other relation's fragments.

Objective 02

Derived Fragments with the Semi-Join

The member inherits the owner's fragmentation through the link: no new predicate is needed.

The rule

$EMP_i = EMP \ltimes PAY_i$

Keep the EMP rows that have a matching title in PAY_i, with EMP's columns only.

→ **Why semi-join, not join?** A join would add sal to EMP. The fragment must keep EMP's own schema.

eno	ename	title	Fragment
E2	M. Smith	Syst. Anal.	EMP_1
E4	J. Miller	Programmer	EMP_1
E5	B. Casey	Syst. Anal.	EMP_1
E8	J. Jones	Syst. Anal.	EMP_1
E1	J. Doe	Elect. Eng.	EMP_2
E3	A. Lee	Mech. Eng.	EMP_2
E6	L. Chu	Elect. Eng.	EMP_2
E7	R. Davis	Mech. Eng.	EMP_2

Objective 02

Why Derived Fragments Make Joins Local

Derived: EMP follows PAY

Site 1 stores PAY_1 and EMP_1; site 2 stores PAY_2 and EMP_2.

Every employee is stored **next to** their pay grade.

$PAY \bowtie EMP = (PAY_1 \bowtie EMP_1) \cup (PAY_2 \bowtie EMP_2)$: two **local** joins, run in parallel.

Independent: EMP cut by eno

$EMP_1 = \sigma$ eno ≤ 'E4', $EMP_2 = \sigma$ eno > 'E4'.

Employees of both pay bands at **both** sites.

Every PAY–EMP join needs rows from the **other** site: four fragment pairs, two of them remote.

Key point: derived fragmentation turns one distributed join into several local joins, one per fragment pair.

Objective 02

Derived Fragments in SQL

```
-- 03_derived.sql (Practice 5, Part B)
1 CREATE TABLE pay1 AS
      SELECT * FROM pay WHERE sal <= 100000;
2 CREATE TABLE emp1 AS SELECT e.* FROM emp e
      WHERE EXISTS (SELECT 1 FROM pay1 p
                    WHERE p.title = e.title);
3 SELECT count(*) FROM emp1 JOIN emp2 USING (eno);
```

1 Primary fragment of the owner, PAY_1.

2 Semi-join: the employees whose title is in PAY_1, EMP's columns only.

← **3 · Check:** the result must be 0. Any employee in both fragments breaks disjointness.

⚠ **Common mistake:** writing the semi-join as **JOIN**. If PAY_1 had two rows with the same title, the join would return an employee twice; EXISTS never does.

Objective 03

A Member with Two Owners: Which One to Follow?

EMP and PROJ are both fragmented. ASG can follow only one of them.

Rule 1 · recommended

Follow the owner whose join is used by **more applications**, weighted by frequency.

If most queries join EMP with ASG, derive ASG from EMP: $ASG_i = ASG \ltimes EMP_i$.

Easy to measure from the workload.

Rule 2

Follow the owner that gives **better join characteristics**.

Smaller fragments, joins that run in parallel, fewer unions.

Hard to estimate in advance; changes as data grows.

Note: rule 1 is a count you can take from the query log. Rule 2 needs a cost model.

Why Not Derive ASG from Both Owners?

EMP and PROJ each have three fragments. What if ASG followed both?

	$PROJ_1$	$PROJ_2$	$PROJ_3$
EMP_1	ASG_{11}	ASG_{12}	ASG_{13}
EMP_2	ASG_{21}	ASG_{22}	ASG_{23}
EMP_3	ASG_{31}	ASG_{32}	ASG_{33}

- Deriving from both gives 3 × 3 = **9** ASG fragments.
- Each piece can sit next to its EMP fragment **or** its PROJ fragment, never both.

→ **Result:** one of the two joins still crosses sites, and there are three times as many fragments.

A member follows the owner it is joined with most often.

Derived fragmentation makes one join local. Choose the join that the applications run most, and accept that the other joins span sites.

- Rule of thumb: count the joins in the query log before you choose an owner.

Objective 04

Correctness of Derived Fragments

The same three rules apply, but two of them are checked through the link to the owner.

Rule	How to check it for $EMP_i = EMP \ltimes PAY_i$
Completeness	Referential integrity: every EMP row has a matching PAY row, so its title appears in some PAY fragment
Reconstruction	$EMP = EMP_1 \cup EMP_2$, as for any horizontal fragmentation
Disjointness	Holds when the join graph is simple: each EMP row joins with exactly one PAY row, because title is PAY's key

Read this row first. An employee with a title missing from PAY is in no fragment.

In practice

A **FOREIGN KEY** from EMP.title to PAY.title guarantees completeness for every future row.

In Practice: Partition-Wise Joins in PostgreSQL

```
-- EXPLAIN with enable_partitionwise_join = on
1 Append
     -> Hash Join (e_1.title = p_1.title)
          -> Seq Scan on emp_high e_1
          -> Seq Scan on pay_high p_1
     -> Hash Join (e_2.title = p_2.title)
          -> Seq Scan on emp_low e_2 · pay_low p_2
```

Critical point: two small joins, high with high and low with low. No cross pair is ever joined.

In practice

Citus co-locates tables distributed on the same column, so their joins run on one worker. This is derived fragmentation in a product.

- With the setting off, PostgreSQL first combines **all** EMP partitions and **all** PAY partitions, then joins them once.

Objective 03

Scenario: Orders in an Online Shop

THE SITUATION

CUSTOMER(cid, name, region) is fragmented by region: **Balkans**, **Turkey**, **Gulf**. PRODUCT(sku, name, category) is fragmented by category.

ORDERS(oid, cid, sku, qty, date) references both.

Customer portal: “my orders”, **50,000** queries a day. Category managers: sales per category, **200** a day.

> “Each region’s data centre should answer its own customers.”

YOUR TASK · PAIRS · 8 MIN

1. Draw the join graph: owners and members.
2. Which owner should ORDERS follow? Why?
3. Write the ORDERS fragments with $\ltimes$.
4. What happens to the category managers’ query?
5. What must hold for completeness?

Objective 04

Check Your Understanding

EMP is fragmented with $EMP_i = EMP \ltimes PAY_i$, but EMP has no foreign key to PAY, and some employees have a title that does not exist in PAY. Which rule fails?

A Completeness

B Reconstruction

C Disjointness

D None: semi-joins are always correct

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Link
A one-to-many relationship between two relations, usually a foreign key.

Owner
The relation on the one side of a link, e.g. PAY for EMP.

Member
The relation on the many side of a link, e.g. EMP for PAY.

Join graph
The relations of a schema with their links drawn from owner to member.

Semi-join R ⋉ S
The rows of R that match some row of S, with R’s columns only.

Derived horizontal fragmentation
Fragmenting a member as member ⋉ owner_i for each owner fragment.

Simple join graph
Each member row joins with exactly one owner row.

Partition-wise join
Joining matching partitions pair by pair instead of all rows at once.

Summary

In this lesson, you learned how to:

- **01** Describe owners, members and the join graph
- **02** Apply the semi-join to derive member fragments
- **03** Decide which owner a member with two owners follows
- **04** Check derived fragments against the three rules

Key takeaway: derive a member as member $\ltimes$ owner$_i$ so that frequent joins are local, follow the owner it joins with most, and enforce the foreign key.

Next week

6 Vertical Fragmentation and Midterm Review

Overview

Practice 5: From Minterms to PostgreSQL Partitions

Setup	Create the database **week05**: PAY 4 rows, EMP 8, PROJ 6, ASG 12.	–
Part A	Minterms for PROJ, six fragments as LIST and RANGE partitions, correctness checks.	Lesson 5-2
Part B	EMP derived from PAY, and a partition-wise join in EXPLAIN.	Lesson 5-3
Part C	Fragment ASG on your own and choose its owner.	Lessons 5-2, 5-3

About **60 minutes** in class, finished at home. You need the course container and the **labs/week05** folder of the kit.

Part A: Six Fragments of PROJ as Partitions

```
-- docker exec -it ce502-postgres psql -U postgres -d lab
1 \i /kit/labs/week05/sql/01_schema.sql
2 \i /kit/labs/week05/sql/02_primary.sql
3 SELECT tableoid::regclass, pno FROM proj_part;
4 INSERT INTO proj_part VALUES
      ('P7', 'Robotics', 90000, 'Tokyo');
5 EXPLAIN SELECT * FROM proj_part
      WHERE loc = 'Paris' AND budget <= 200000;
```

1. Create week05 and load the data.
2. Build the six fragments and run the checks.
3. See which fragment each project went to.

← **4 · Critical:** this insert fails. Which rule breaks? Fix it with a **DEFAULT** partition.

5. Only **proj5** is scanned: partition pruning.

</> First do Exercise 1 on paper, then check it on your laptop: **python3 labs/week05/python/minterms.py**

Parts B and C

Part B: EMP Follows PAY

```
-- in psql, connected to week05
1 \i /kit/labs/week05/sql/03_derived.sql
2 SELECT * FROM emp1; -- EMP ⋉ PAY1
3 SET enable_partitionwise_join = off;
4 EXPLAIN SELECT ename, sal FROM emp_part
      JOIN pay_part USING (title);
5 SET enable_partitionwise_join = on;
```

1 Fragment PAY by salary; derive EMP with EXISTS; run the checks.

2 Four employees: E2, E4, E5, E8.

3–4 Setting off: one join over all partitions.

← **5 · Critical:** then run the EXPLAIN of line 4 again. Now each emp partition joins only its own pay partition.

Part C, on your own: fragment ASG (worksheet Exercise 2), and decide in 5–8 lines whether ASG should follow EMP or PROJ.

Deliverables

What to Hand In

One PDF before the Week 6 class.

Report item	Points
Part A: Exercise 1 on paper (minterms, implications, fragments, rules)	3
Part A: output of 02_primary.sql explained, incl. the Tokyo error and pruning	2
Part B: derived fragments and the two EXPLAIN plans compared	2
Part C: ASG fragments in SQL with the reconstruction check	2
Part C: owner choice for ASG, 5–8 lines	1
Total	10

Key point: every output in the report needs one sentence that names the rule or idea it proves.



Papers to Read

Two required papers and three optional ones on partitioning design, from minterm predicates to workload graphs.

Before the Week 6 class

Half-page note per required paper

Required

Required Reading for Week 6

Horizontal Data Partitioning in Database Design

S. Ceri, M. Negri, G. Pelagatti · ACM SIGMOD 1982, pp. 128–136

Where minterms, completeness and minimality come from. Read the introduction and the sections on primary and derived partitioning.

Schism: a Workload-Driven Approach to Database Replication and Partitioning

C. Curino, E. Jones, Y. Zhang, S. Madden · PVLDB 3(1), 2010, pp. 48–57

Partitions cut from a graph of real transactions. Read the introduction, the overview and the graph sections.

Bring a half-page note per paper: the problem, the key idea, and its link to this week.

Optional

Going Further

Skew-Aware Automatic Database Partitioning in Shared-Nothing, Parallel OLTP Systems
A. Pavlo, C. Curino, S. Zdonik · ACM SIGMOD 2012, pp. 61–72

Choosing partitioning keys automatically, and when to replicate a table instead.

Citus: Distributed PostgreSQL for Data-Intensive Applications
U. Cubukcu et al. · ACM SIGMOD 2021, pp. 2490–2502

Co-location of tables: derived fragmentation inside a product.

Principles of Distributed Database Systems, 4th ed.
M. T. Özsu, P. Valduriez · Springer 2020, chapter 2

The reference for every definition of Weeks 5 and 6.

Required

Fragments Come From the Applications' Predicates

Ceri, S., Negri, M., & Pelagatti, G. (1982). Horizontal data partitioning in database design. In Proceedings of the 1982 ACM SIGMOD International Conference on Management of Data (SIGMOD '82) (pp. 128–136). ACM. https://doi.org/10.1145/582353.582376

THE PROBLEM

A relation used by applications at several sites can be split into groups of rows. The authors ask how to choose these groups so that each application reads the rows it needs and few others.

KEY IDEAS

01 Simple predicates come from the applications, e.g. one attribute compared with a value

02 A minterm joins all simple predicates, each as-is or negated; each minterm defines a fragment

03 Derived partitioning splits a relation by the fragments of a related relation it is joined with

Key term: Complete predicate set — Any two rows in the same fragment are accessed with equal probability by every application

Required

Complete and Minimal Predicates Give Good Fragments

WHAT TO TAKE AWAY

1. With a complete set, no application reads only part of a fragment, so its access cost is predictable
2. Minimality drops predicates that split rows which no application treats differently
3. The same method fits data across sites, across primary and secondary memory, or across devices

THIS WEEK

Lesson 5-2 uses its terms: simple predicates, minterms, completeness and minimality. In Practice 5 Part A you derive the six PROJ minterms and build them as partitions.

LIMITATION

Assumes the designer knows every important application in advance. It gives no way to measure access frequencies or to adapt when use changes.

For your half-page note: Why do Ceri et al. require that rows in one fragment are accessed uniformly? What goes wrong when a predicate set is not complete?

WEEK 5 · REQUIRED READING 2 · CURINO ET AL. 2010

Required

Partition the Graph of Real Transactions

Curino, C., Jones, E., Zhang, Y., & Madden, S. (2010). Schism: A workload-driven approach to database replication and partitioning. Proceedings of the VLDB Endowment, 3(1–2), 48–57. https://doi.org/10.14778/1920841.1920853

THE PROBLEM

In an OLTP database on many machines, transactions that touch several partitions are slow. Hash and range partitioning often cannot avoid them, above all with n-to-n relations. Schism learns a better partitioning from the workload.

KEY IDEAS

01 Graph: one node per tuple; an edge joins two tuples used by the same transaction

02 METIS cuts the graph into k balanced parts with the least total weight of cut edges

03 A rarely updated tuple can be replicated: its node becomes a star of replica nodes

Key term: Distributed transaction — A transaction that accesses more than one partition, so it needs two-phase commit and extra messages

Required

Fewer Distributed Transactions, Explained by Predicates

WHAT TO TAKE AWAY

1. In a MySQL test, distributed transactions had about half the throughput and double the latency
2. A decision tree turns per-tuple labels into range predicates, e.g. TPC-C split by warehouse id
3. On Epinions.com, 2 partitions: 4.5% distributed transactions, against 6% for a manual design

THIS WEEK

Lesson 5-1's slide The Price of Fragmentation is Schism's measure: transactions that span sites. Its decision-tree rules are predicates, like the minterms of Practice 5 Part A.

LIMITATION

Needs a representative trace and a fixed number of partitions. Large graphs need sampling, and a changing workload means partitioning again.

For your half-page note: Schism ends with range predicates, as Ceri et al. do. Why does it need this step if the graph partition already places every tuple?

Optional

Search Many Designs With a Skew-Aware Cost Model

Pavlo, A., Curino, C., & Zdonik, S. (2012). Skew-aware automatic database partitioning in shared-nothing, parallel OLTP systems. In Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD '12) (pp. 61–72). ACM. https://doi.org/10.1145/2213836.2213844

THE PROBLEM

Parallel OLTP systems scale only if most transactions use one partition and the load is even. Real applications add stored procedures, complex schemas and load that shifts over time. Can a tool find a good design?

KEY IDEAS

01 For each table: partition it on a column or replicate it; secondary indexes can be replicated

02 Each stored procedure gets a routing parameter, so it starts where most of its data is

03 Large-neighbourhood search: reset part of the best design, then search locally for a cheaper one

Key term: Temporal skew — Load that looks even over a whole day but is concentrated on a few partitions at any one moment

Optional

Distribution and Skew Must Be Weighed Together

WHAT TO TAKE AWAY

1. In H-Store, only 10% distributed transactions cut throughput by about 2× at 64 partitions
2. Sending just 5% extra load to one partition reduced throughput by more than 3× in their test
3. Horticulture's designs ran 1.3–4.3× faster than Schism's and 1.1–16.3× faster than a baseline

THIS WEEK

Lesson 5-2 notes that the optimal design is too hard to find, so tools use heuristics; this is one. In Practice 5 Part C you choose ASG's owner by hand.

LIMITATION

Built and tested only on H-Store, a main-memory system driven by stored procedures. It assumes the sample trace is representative.

To think about: Putting every row on one node gives no distributed transactions. Why is that still a bad design, and how does the cost model avoid it?

Optional

Co-Located Shards Make Distributed Joins Local

Cubukcu, U., Erdogan, O., Pathak, S., Sannakkayala, S., & Slot, M. (2021). Citus: Distributed PostgreSQL for data-intensive applications. In Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21) (pp. 2490–2502). ACM. https://doi.org/10.1145/3448016.3457551

THE PROBLEM

Many PostgreSQL applications outgrow a single server, and moving them to a new distributed database is costly. The authors describe how Citus scales PostgreSQL across many servers while keeping its features.

KEY IDEAS

01 Citus is a PostgreSQL extension: a coordinator plans queries, and workers store the shards

02 A distributed table is split into hash shards on a distribution column chosen by the user

03 Reference tables are small tables copied in full to every worker

Key term: Co-location — Storing the shards of several tables that cover the same distribution values on the same worker

Optional

Four Workload Patterns Benefit From Scaling Out

WHAT TO TAKE AWAY

1. Four patterns: multi-tenant, real-time analytics, high-performance CRUD and data warehousing
2. Joins and foreign keys between co-located tables run on each worker without moving data
3. Transactions across workers use two-phase commit, with distributed deadlock detection

THIS WEEK

Lesson 5-3's partition-wise join slide is co-location inside one server. In Practice 5 Part B, emp_low joins only pay_low; in Citus each worker would run one such join.

LIMITATION

Written by the developers about their own product, with their own benchmarks. Choosing a good distribution column is left to the user.

To think about: An app's tables are distributed by tenant id. Which tables would you co-locate with them, and which would you make reference tables?

Optional

Design Is Fragmentation Followed by Allocation

Özsu, M. T., & Valduriez, P. (2020). Distributed and parallel database design. In Principles of distributed database systems (4th ed., pp. 33–89). Springer. https://doi.org/10.1007/978-3-030-26253-2_2

THE PROBLEM

A distributed database must decide what to store at each site. Chapter 2 of the standard textbook gives the methods: how to fragment relations, how to check the fragments, and where to place them.

KEY IDEAS

01 Horizontal fragments come from minterms of simple predicates; derived ones use semi-joins

02 Vertical fragments come from attribute affinity, clustered by the bond energy algorithm

03 Every fragmentation must be complete, reconstructible and disjoint

Key term: Minterm predicate — A conjunction of all simple predicates, each as it is or negated; n predicates give 2^n minterms

Optional

From Classic Fragmentation to Workload-Aware Design

WHAT TO TAKE AWAY

1. Allocation places fragments at sites to lower total cost; the optimal allocation is NP-hard
2. Parallel systems often use simpler round-robin, hash or range partitioning instead
3. Workload-aware methods, such as Schism, and adaptive methods follow how the data is used

THIS WEEK

The reference for Lessons 5-1 to 5-3: the three rules, the minterm algorithm and semi-join derivation. Practice 5 uses its EMP, PAY, PROJ and ASG example schema.

LIMITATION

A textbook chapter: it explains each method with small examples but gives little evidence of how well the methods work on large, real workloads.

To think about: The chapter treats fragmentation and allocation as separate steps. When could doing them one after the other give a poor design?

Questions

Reading Questions

Try to answer each in two or three sentences before the next class.

Q1	Why are fragments built from minterms of **all** predicates rather than from each predicate on its own?	Ceri et al.
Q2	What are the nodes and edges of Schism’s graph, and what does a good cut minimise?	Curino et al.
Q3	Schism starts from a workload trace, the minterm method from the designer’s predicates. When would you prefer each?	Both papers
Q4	Which idea from Lesson 5-3 does Citus call co-location, and what do Pavlo et al. replicate instead of partitioning?	Optional papers

WEEK 5 · WRAP-UP

This Week’s Work

Practice 5

Lab report

Minterms, partitions, derived fragments and ASG. One PDF, 10 points.

Due: before Week 6

Quiz 5

Horizontal fragmentation

Lessons 5-1 to 5-3: 16 multiple-choice questions and one design question, 30 minutes.

Due: before Week 6

Reading

Required papers

Ceri et al. 1982 and Schism. A half-page note for each paper.

Bring to Week 6

Next week: cutting by columns. Vertical fragmentation, the bond energy algorithm, and the midterm review.